

SECTIUNEA 1 — Feedback

Realizarea acestei teme a reprezentat un exercițiu amplu și util pentru consolidarea abilităților de programare paralelă și înțelegerea modului în care pot fi construite pipeline-uri eficiente pentru procesarea unor volume mari de date. Tema presupune parcurgerea unui flux complex, de la citirea fișierelor JSON până la generarea unor rapoarte finale, ceea ce a necesitat o structurare atentă a etapelor și o abordare metodică. Una dintre principalele provocări a fost coordonarea firelor de execuție astfel încât fiecare etapă să fie complet paralelizată acolo unde este posibil, dar în același timp sincronizată în punctele critice. Eliminarea duplicatelor a necesitat o vizibilitate globală asupra articolelor, iar impunerea utilizării unui singur set fix de thread-uri a făcut ca designul final să fie cu atât mai important pentru obținerea corectitudinii și scalabilității. Tema a fost solicitantă și a necesitat mai multe sesiuni de implementare și testare, inclusiv ajustări ale mecanismelor de sincronizare și optimizarea modului de procesare a textului. Totuși, exercițiul a fost valoros și relevant, întrucât reproduce cu fidelitate provocări întâlnite în procesarea reală a datelor la scară mare.

SECTIUNEA 2 — Strategia de paralelizare

Soluția implementată folosește un model de tip Map–Reduce distribuit peste un set fix de thread-uri create simultan la începutul execuției, conform cerințelor temei. Toate thread-urile participă la întregul pipeline: citirea fișierelor, colectarea informațiilor, filtrarea articolelor valide și acumularea statisticilor locale, după care thread-ul principal realizează reducerea globală și scrierea fișierelor de ieșire. Fișierele sunt distribuite thread-urilor printr-o structură thread-safe de tip `ConcurrentLinkedQueue`, din care fiecare thread extrage independent câte un path. Această abordare asigură o distribuție dinamică a sarcinilor și elimină problemele legate de diferențele de dimensiune dintre fișiere. Folosind biblioteca Jackson, fiecare thread parsează fișierul JSON în obiecte `Article`, pe care le stochează într-un buffer local.

Pentru identificarea duplicatelor, fiecare thread actualizează în paralel două structuri globale `ConcurrentHashMap` care contorizează aparițiile fiecărui UUID și titlu. Aceste operații sunt eficiente datorită segmentării interne a structurii și permit acumularea datelor la nivel global fără blocări majore. Odată terminată etapa de citire, toate thread-urile se sincronizează printr-o `CyclicBarrier`, garantând că registrele globale sunt complete înainte de deduplicare. După sincronizare, fiecare thread își parcurge bufferul local și păstrează doar articolele ale căror UUID și titlu apar o singură dată în întreg datasetul.

Procesarea articolelor valide include extragerea autorilor, limbilor, categoriilor și a cuvintelor relevante din textele în limba engleză. Toate aceste statistici sunt menținute

în structuri locale fiecărui thread, reducând complet nevoia de sincronizare în faza de prelucrare. Textul este procesat prin conversie la litere mici, segmentare în cuvinte, eliminarea caracterelor non-alfabetice și filtrarea stop-word-urilor. După finalizarea acestei etape, o a doua barieră CyclicBarrier asigură faptul că toate thread-urile și-au încheiat complet partea de Map înainte ca reducerea globală să înceapă.

Thread-ul principal realizează apoi reducerea finală prin combinarea structurilor locale într-un set de hărți globale care conțin statisticile finale. Acest proces este secvențial și reprezintă partea firească de final a unui pipeline Map–Reduce. Designul soluției este corect deoarece deduplicarea este deterministă și realizată cu vizibilitate completă asupra datelor, iar structurile locale previn apariția condițiilor de cursă. În ceea ce privește eficiența, citirea paralelă maximizează utilizarea procesorului, sincronizarea globală este menținută la minimum, iar distribuirea echilibrată a sarcinilor contribuie la o scalabilitate bună a implementării.

SECȚIUNEA 3

CPU = AMD RYZEN 9 HX

NR NUCLEE = 24

RAM = 32 GB

SO = Windows 11 Pro

Versiunea de java = 17.0.17

Am folosit data set ul de la test_1 din checkerul local, adica am rulat comanda make run ARGS="P ../checker/input/tests/test_1/articles.txt ../checker/input/tests/test_1/inputs.txt"

Unde P reprezinta nr de thread uri.

Nr. Threaduri	Time run 1	Time run 2	Time run 3	Time run 4	Average
1	3.12	3.14	3.07	3.18	3.12
2	2.22	2.19	2.23	2.25	2.22

3	1.85	1.83	1.91	1.80	1.84
4	1.61	1.63	1.58	1.67	1.62
8	1.52	1.51	1.53	1.49	1.51

Pt 1 thread:

$$S(1) = 3.12 / 3.12 = 1.00$$

$$E(1) = 1.00 / 1 = 1.00$$

Pt 2 thread uri:

$$S(2) = 3.12 / 2.22 \approx 1.405$$

$$E(2) = 1.405 / 2 \approx 0.702$$

Pt 3 thread uri:

$$S(3) = 3.12 / 1.84 \approx 1.696$$

$$E(3) = 1.696 / 3 \approx 0.565$$

Pt 4 thread uri:

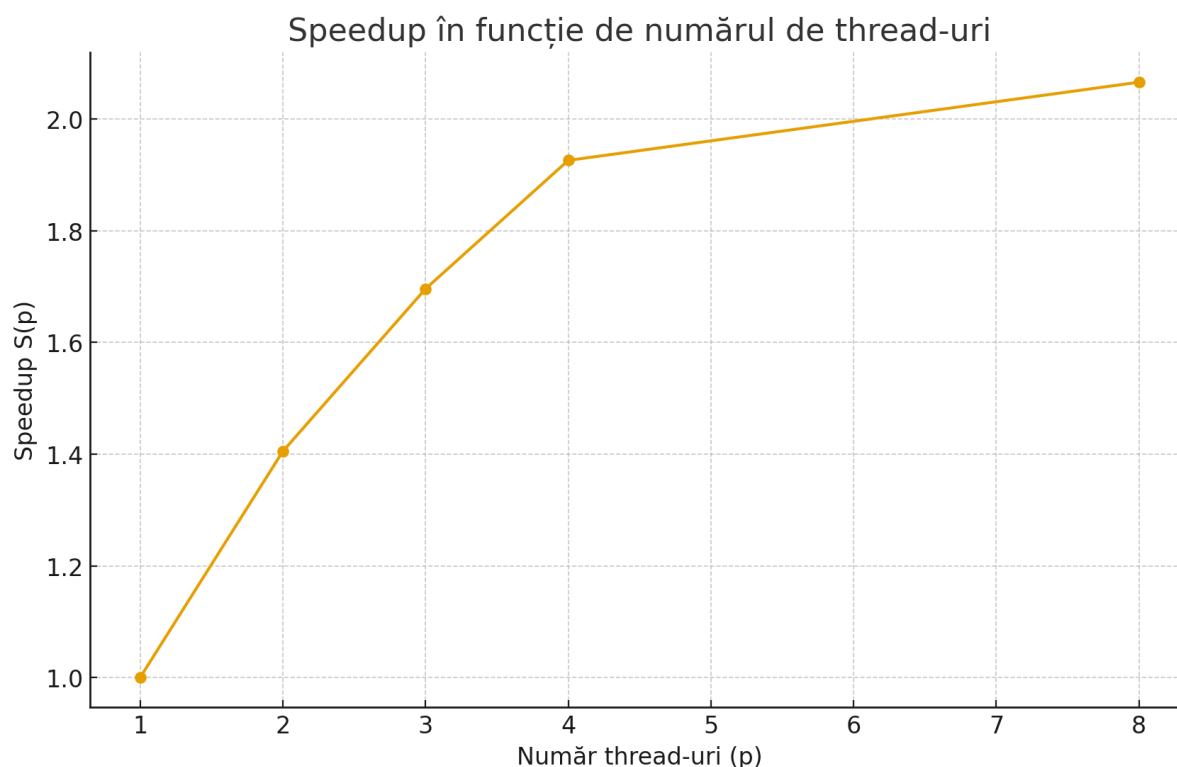
$$S(4) = 3.12 / 1.62 \approx 1.926$$

$$E(4) = 1.926 / 4 \approx 0.481$$

Pt 8 thread uri:

$$S(8) = 3.12 / 1.51 \approx 2.066$$

$$E(8) = 2.066 / 8 \approx 0.258$$



Acesta e graficul cu Speed up in functie de nr de thread uri.

Performanța programului crește vizibil atunci când mărim numărul de thread-uri, însă creșterea nu este una liniară și, după un anumit punct, se vede clar că beneficiile încep să scadă. Cea mai mare diferență apare între 1 și 2 thread-uri, unde timpul de execuție scade destul de mult, ceea ce era de așteptat, fiind prima împărțire reală a muncii. De la 2 la 3 și apoi la 4 thread-uri, performanța continuă să se îmbunătățească, dar mai puțin spectaculos. La 8 thread-uri, timpul scade foarte puțin față de 4, iar eficiența pe fiecare thread este mult mai slabă.

Acest comportament apare din mai multe motive. În primul rând, datasetul nu este foarte mare, iar numărul de fișiere este limitat. Când sunt prea multe thread-uri în comparație cu numărul de fișiere, unele thread-uri ajung să nu mai aibă aproape deloc muncă sau așteaptă după celelalte. În al doilea rând, chiar dacă folosim structuri thread-safe eficiente, tot există un anumit overhead atunci când mai multe thread-uri accesează resurse comune, cum ar fi registrele globale pentru duplicate. De asemenea, barierele de sincronizare pot încetini execuția atunci când thread-urile nu ajung exact în același timp la punctul de sincronizare.

Din toate aceste motive, numărul optim de thread-uri pentru sistemul și datasetul testat pare să fie 4. Acolo se obține un timp foarte bun raportat la resursele folosite, iar eficiența este încă rezonabilă. Folosirea a 8 thread-uri reduce timpul doar puțin, dar

scade eficiența, ceea ce arată că programul nu reușește să folosească în mod real toate firele suplimentare pentru un workload atât de mic. Pe scurt, paralelizarea ajută mult, dar doar până la un anumit punct, după care sistemul începe să fie limitat mai mult de structura datelor și de sincronizările necesare decât de puterea procesorului.